ME2605 / DDA6204 · Neural Net Training · Fall 2026

# Lecture 7: ResNet and Landscape

**Ruoyu Sun**

# Today's Plan

01 Part 1

## Skip Connections and ResNet

Deep plain nets train worse than shallow ones.

The skip connection fixes optimization.

02 Part 3

## Initialization and Normalization & ResNet

Signal propagation in ResNet.

Different combinations of skip connection and normalization

03 Part 4

## Loss Landscape

Visualize the loss landscape.

How skip connection and width helps landscape.

# Learning Goals

By the end of this lecture, you should be able to…

| | |
|---|---|
| 01 | Explain why adding layers can hurt performance, and how people come up with skip connection |
| 02 | Compute signal propagation in MLP with skip connection<br>Compare different ways to place normalization, ReLU and skip connection. |
| 03 | Explain how to draw the loss landscape; and tell the benefit of width and skip connection. |

// PART 01

# Skip Connections and ResNet

01

Deeper nets got harder to train.
What went wrong, and how a skip connection fixes it.


01 Skip Connections and ResNet
02 Practical ResNet
03 Initialization & Normalization of ResNet
04 Loss Landscape

# ResNet in Numbers

**347,000+**

Google Scholar citations (Oct 2026)

**Deep Residual Learning for Image Recognition**

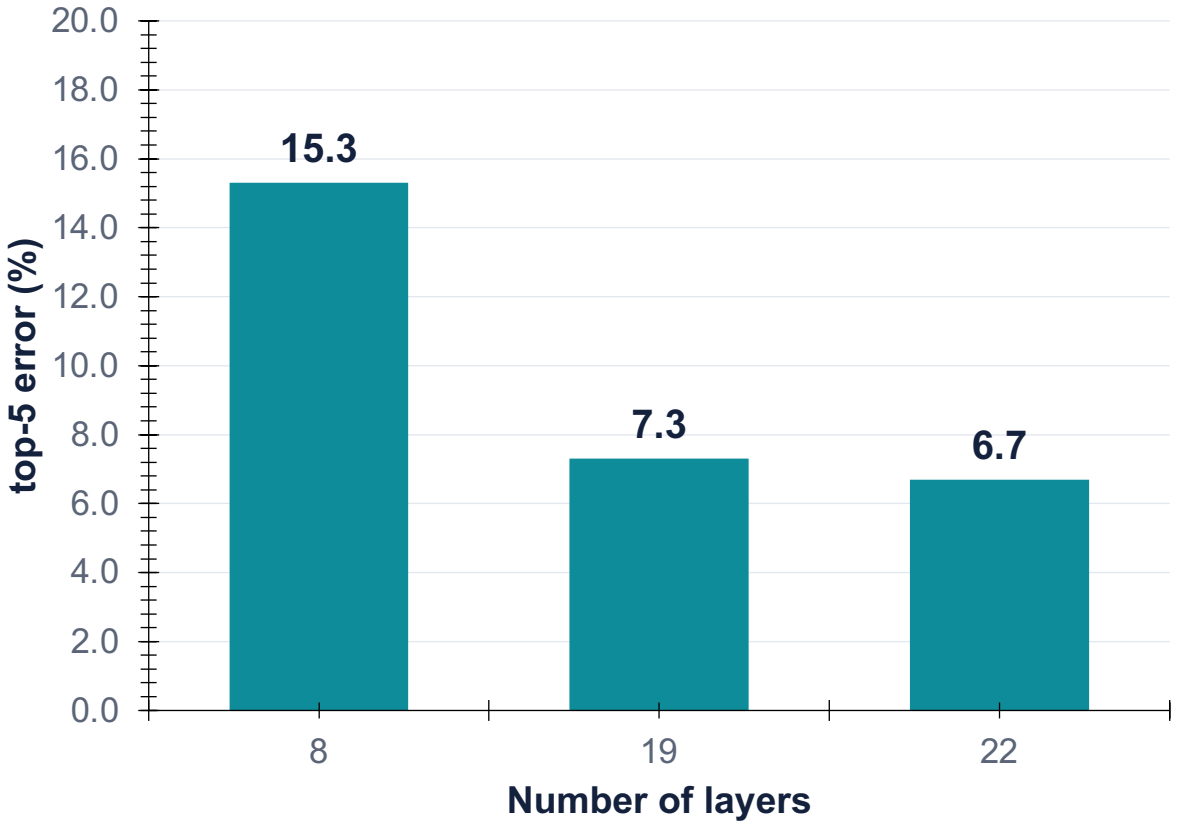
He, Zhang, Ren, Sun · CVPR 2016

**No. 1**

- The most-cited scientific paper of the 21st century (Nature analysis, April 2025)
- Skip connections are in every transformer block

TERMS **CVPR**: the main computer-vision conference · **transformer**: the architecture behind large language models

# Historical Background: Go Deeper




AlexNet → VGG → GoogLeNet:
deeper nets, lower ImageNet error.

**Trend: go deeper.**

But even with smart initialization and BatchNorm,

training more than 20–30 layers was hard.

# Why?

TERMS **AlexNet, VGG, GoogLeNet**: convolutional nets (CNNs) that read images · **ImageNet**: 1.28M photos, 1000 classes

# Two Paths to Train Deeper Nets

**PROBLEM** In a deep net the signal may vanish or explode (variance analysis)

PATH A

## Change the parameters architecture fixed



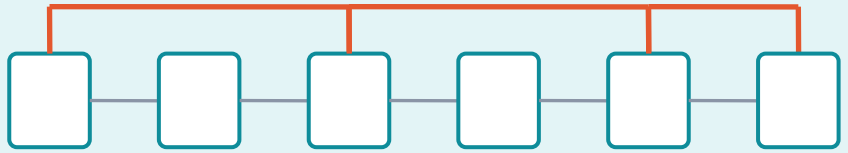
same plain layers, W ← orthogonal init

- Dynamical isometry: orthogonal initialization
- Trains 10,000 layers, no skips, no BN (Xiao et al. 2018)
- Trainable, but accuracy seems not good enough

**STILL ONGOING RESEARCH**

PATH B

## Change the architecture add skip connections



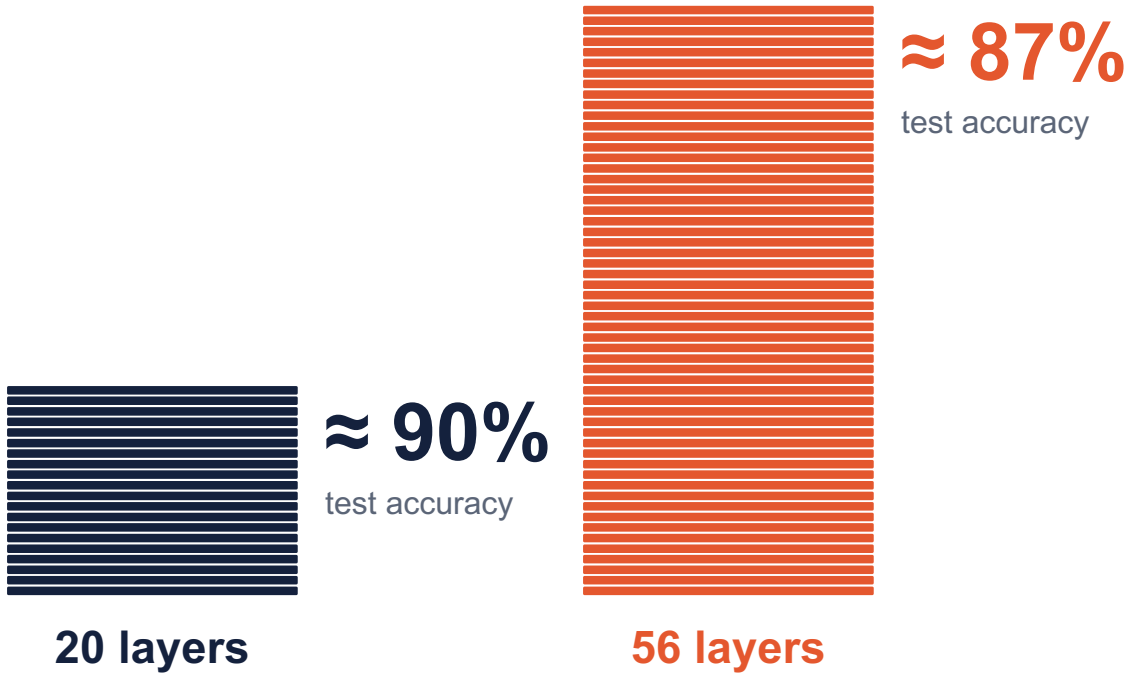
same layers + skip connections

- Skip paths ⇒ ResNet (He et al. 2016)
- Became the standard way to go deep
- Part 1: why does it work?

**HISTORY CHOSE THIS PATH**

# The Fact: 56 Layers Are Worse Than 20

He et al. trained two **plain** nets (no skip connections, with BN) on CIFAR-10.
Same recipe, only the depth differs.




**36 more layers, lower accuracy**

Same data, same training recipe.

Only difference: 36 more layers.

Yet the deeper net is **worse**.

## Why?

FROM PAPER He et al. 2016, Fig. 1 · plain CNNs on CIFAR-10 · accuracies read off the test-error curves

TERMS **CIFAR-10**: 60,000 small (32×32) color images, 10 classes

VOTE · A OR B



# Diagnosis Step 1: Training or Test?

The 56-layer net has a higher **test** error. Where does the extra error come from?

**A. Larger training error**

It fits the training data worse

**B. Larger generalization gap**

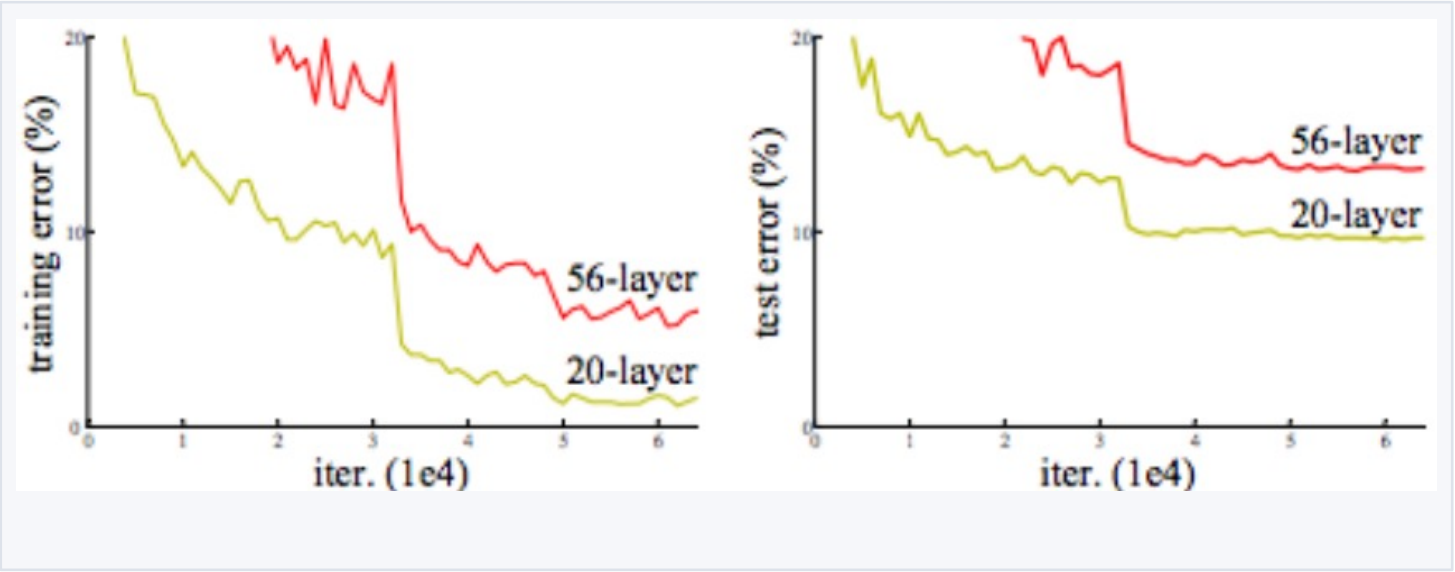
Similar training error, larger test error

Natural guess from statistical learning: **overfitting**, i.e. B.

More layers → more parameters → overfit?

You may debate for a day … or simply check the training error.

# Train or Test?




FROM PAPER He et al. 2016 · CNNs on CIFAR-10 · left: training error, right: test error

**Higher training error too.**

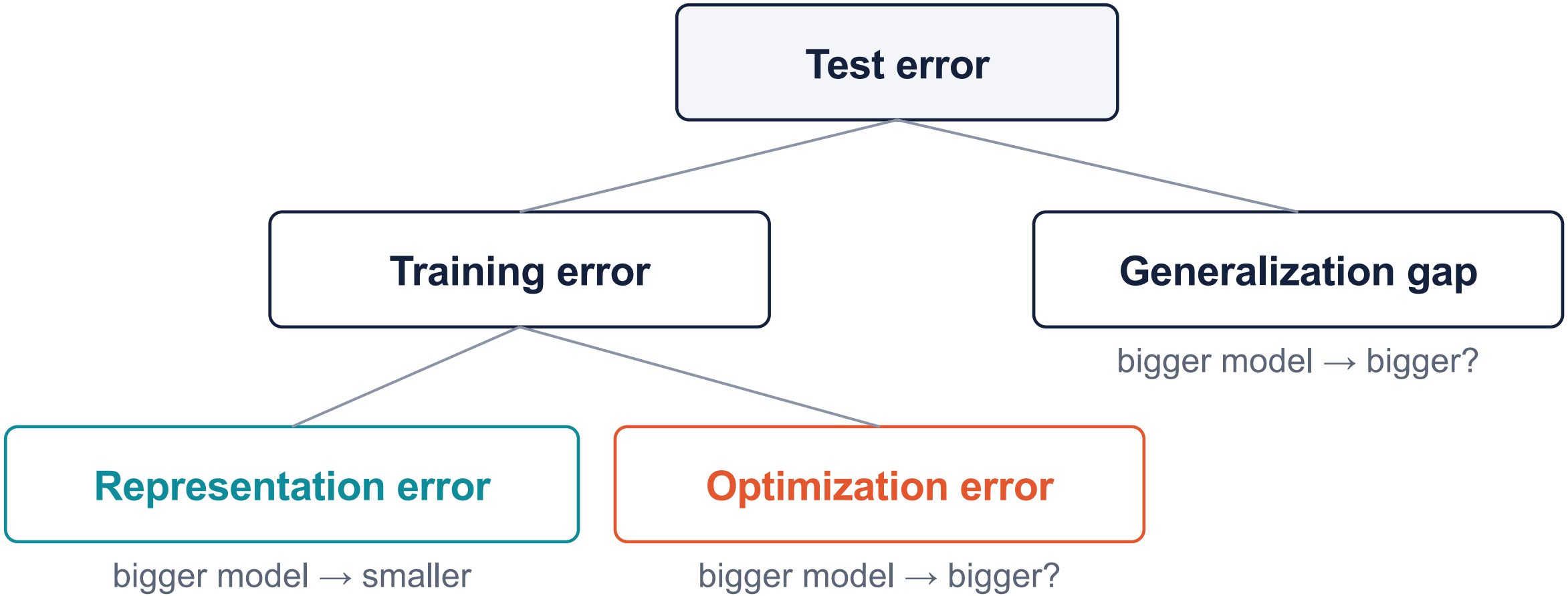
The 56-layer net is worse on the **training set itself**, during the whole run.

Overfitting cannot explain a higher training error.

> Step 1: the problem is on the **training** side.

TERMS **iter. (1e4)**: training steps, in units of $10^4$

# Decomposing the Error



COMPUTE



# Your Turn: Train or Test?

Two nets, same data, same budget, both 20 layers.

A: train error 1%, test error 8%. B: train error 5%, test error 7%.

**Two minutes: compute both train–test gaps, then say which net has an optimization problem.**

Answer. The gaps are 7 and 2, so the gap itself does not separate them: A fits the training set almost perfectly, B cannot fit it at all. B is the one with an optimization problem.

// DISCUSS

VOTE



# Pause: Representation or Optimization?

The 56-layer net has a **higher training error** than the 20-layer net.

## Vote: representation error or optimization error?

How could we tell?

**Recall Lecture 1**

In general the two are hard to separate: we do not know the best achievable training error.

**This case is easier**

The two nets differ only in depth.

Hint: what can a 56-layer net copy?

# Representation or Optimization? A Construction

**Construction:** any 20-layer ReLU MLP + 36 identity layers = a 56-layer net computing the **same function**.




36 extra layers, each set to the identity. ReLU layer with W = I, b = 0: output = input (its input is already ≥ 0).

Notation: $E_k$ = training error after training $k$-layer net; $E_k^*$ = best training error with $k$ layers.

$$E_{56}^* \leq E_{20}^*$$

Every 20-layer function is a 56-layer function ⇒ **representation error ruled out**

$$(E_{56} - E_{56}^*) - (E_{20} - E_{20}^*) \geq E_{56} - E_{20} > 0$$

optimization error (56) − optimization error (20) ≥ observed gap > 0

The 56-layer net has a **larger optimization error**, by at least the observed gap.

# From Diagnosis to a Fix

### So far: a diagnosis

The deep plain net does **not reach a good solution**.

GD fails to converge, or converges to a bad point.

### But: no cure yet

Knowing that GD fails does not tell us **how to fix it**.

Here the analysis and the fix do not match directly: a long-standing challenge.

### Next: a specific analysis

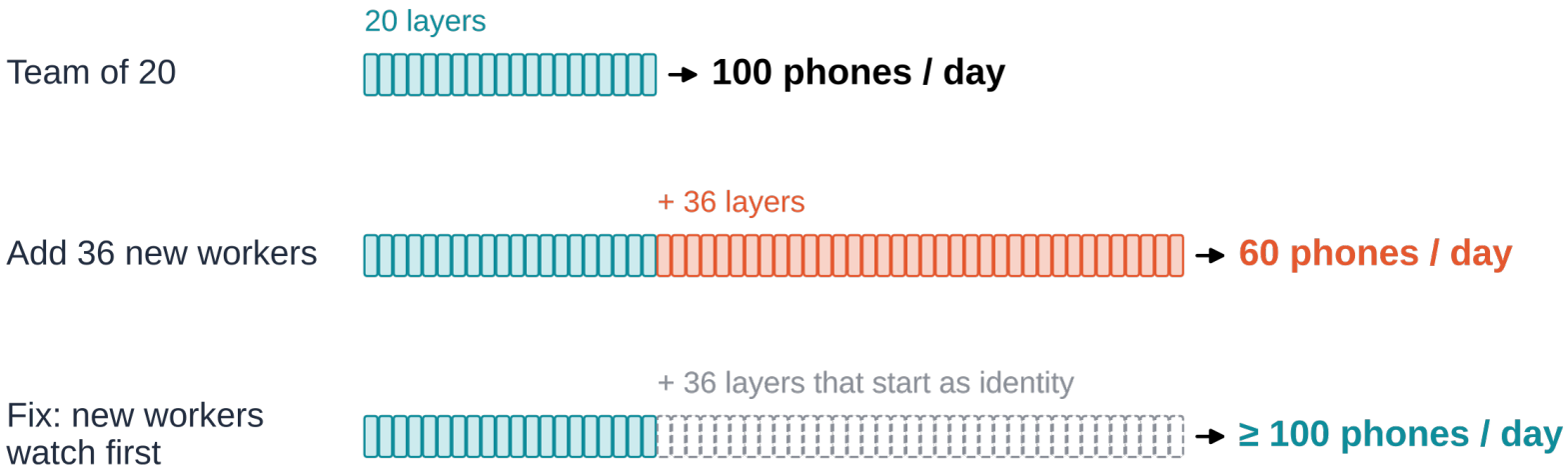
For this problem: what should the **36 extra layers** do?

This leads to a fix.

Problem-specific thinking: the extra layers should not hurt a net that already works.

# Rethinking the Role of Extra Layers

**Rethink:** we have 36 extra layers, but they do not help; they hurt!



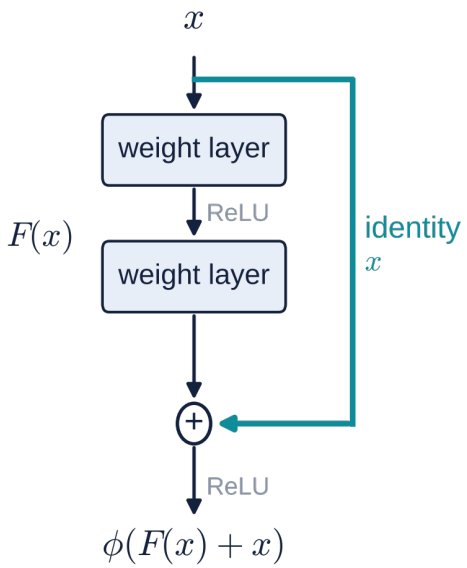

An analogy: new workers put into the line on day one disturb it.

> Solution: ask them to **do nothing first**; join only when they have learned enough.

# How? Put the Identity on the Side

**Obstacle:** a random layer $\phi(Wz)$ is far from the identity.

**Fix:** keep the random layer; add an identity mapping on the side.




**ResNet module**, simplest version:

$$z^{\ell} = \phi(W^{\ell} z^{\ell-1}) + z^{\ell-1}, \quad \ell = 1, \ldots, L$$

General version:

$$y = F(x, W^{\ell}) + x$$

# Dimension Mismatch

If a block changes the width, $x \in R^d$ but $F(x) \in R^{d'}$ with $d' > d$: we cannot add them.

**Two tricks:**

### 1. Linear projection on the shortcut

$$y = F(x, W) + W_s\, x, \quad W_s \in R^{d' \times d}$$

### 2. Identity with zero padding

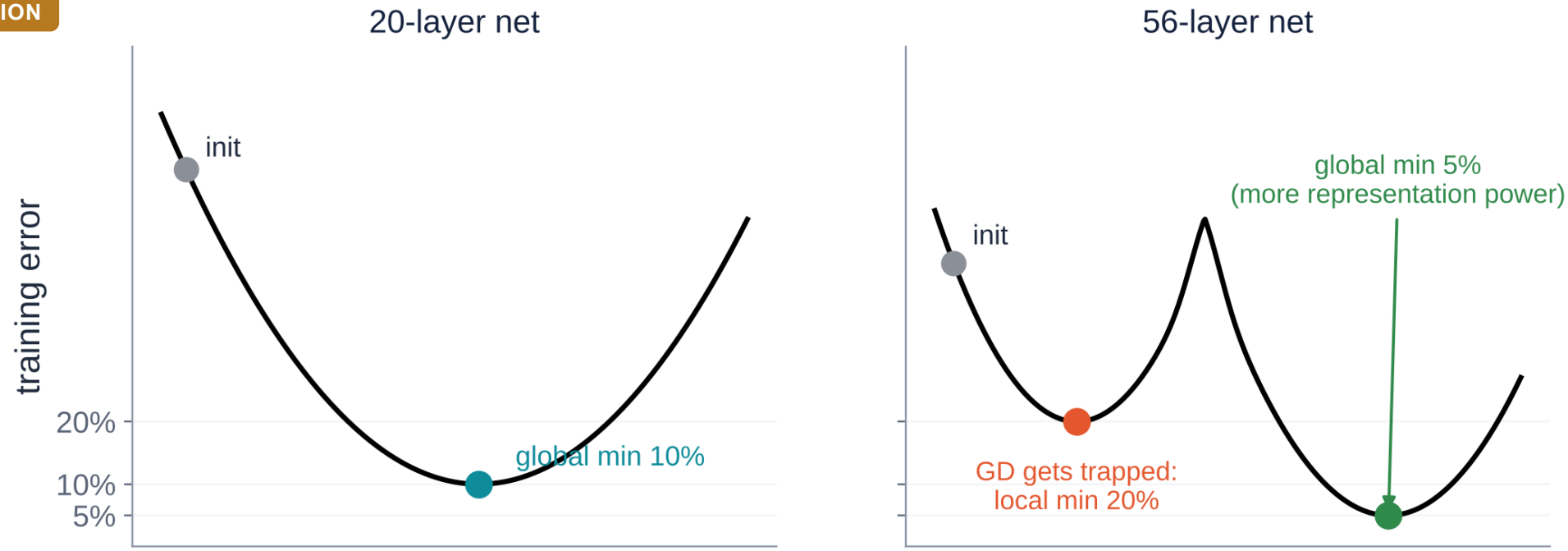
Append $d' - d$ zeros to $x$, then add

No extra parameters

In CNNs the width is the number of channels.

# Representation Power and Global Min

ILLUSTRATION

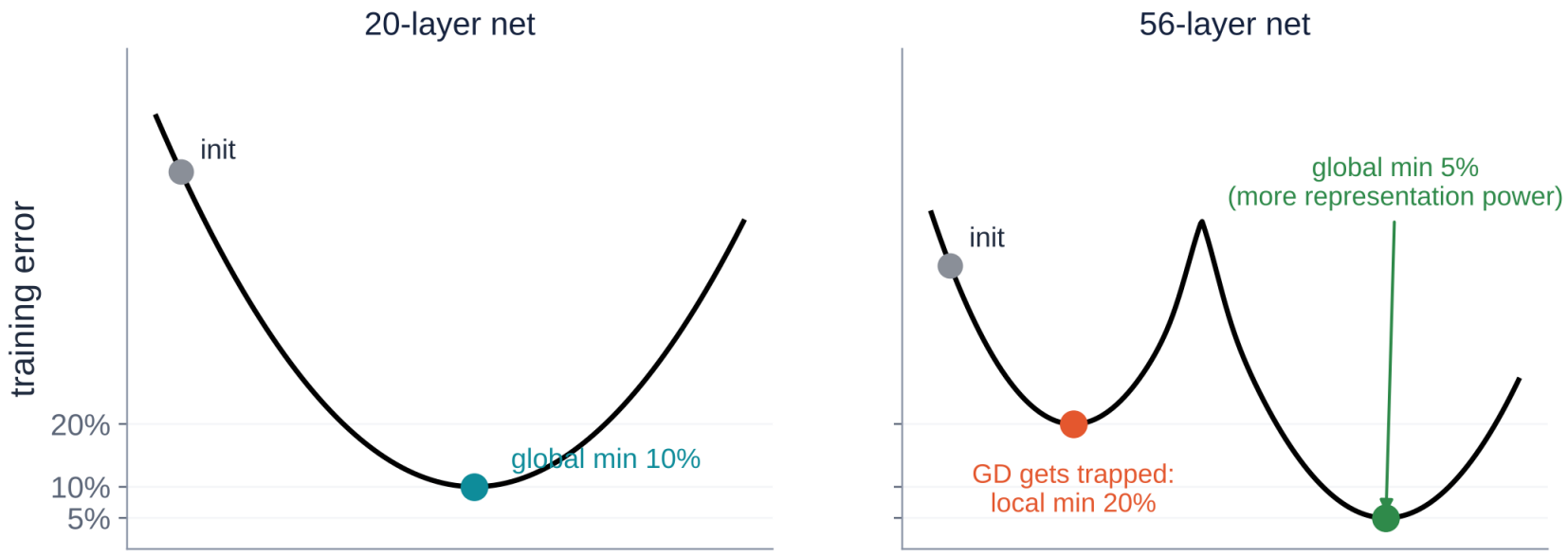



- Global min of the 20-layer net: 10% error
- 36 more layers add representation power: the 56-layer global min has 5% error
- But GD from a random init ends in a local min with **20%** error.

**[Disclaimer: There is no rigorous proof that GD gets trapped at local-min for 2—layer net; here is just an illustration.]**

# Local Min and Local Representation Power

ILLUSTRATION




Does tuning the learning rate help? Does changing the initialization variance help?

Not ruled out: schedules and initialization variance can change signal propagation. Part 2

**Change the environment: change the landscape**

**Change your position: start near the old min**

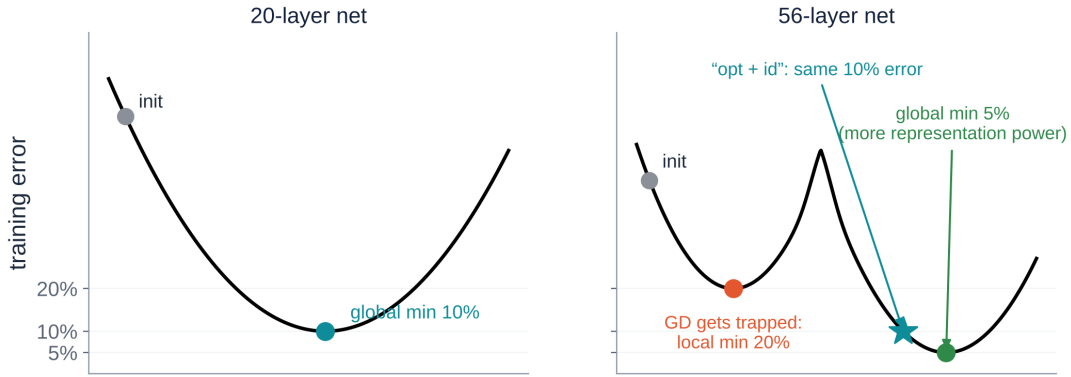
# Combination: A First Solution

Candidate solution 1: train the 56-layer net starting from




Starting error = 10% (same as the 20-layer net); training can only lower it.

ILLUSTRATION




→ **Solution 1:** Start at 10% error. For full-batch GD on a smooth loss, a sufficiently small step ensures non-increasing loss.

# Modification of Solution 1?

Solution 1 is a thought experiment.

Standard ResNet training does not use this two-stage procedure.

**SOLUTION 1**

**Stage 1: train 20 layers** — **Stage 2: add 36 identity layers, train all 56**

two stages · needs the 20-layer solution first

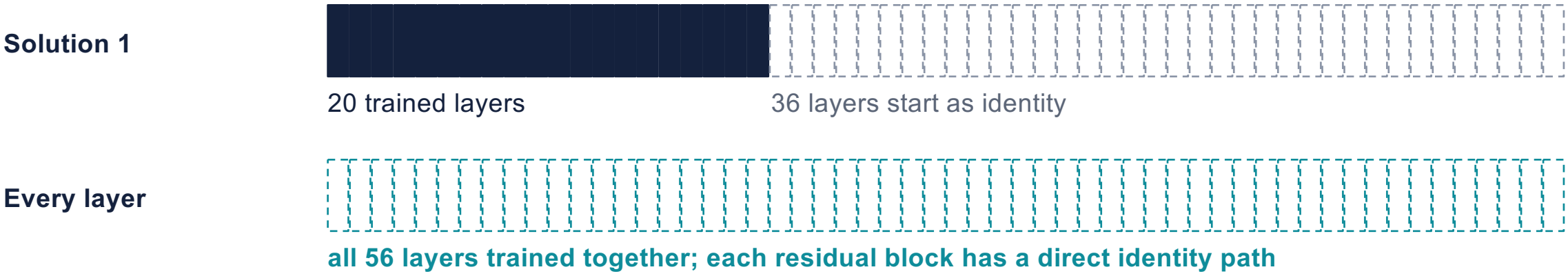
**Hope**

**One run: all 56 layers trained together, from a random initialization**

**what we want**

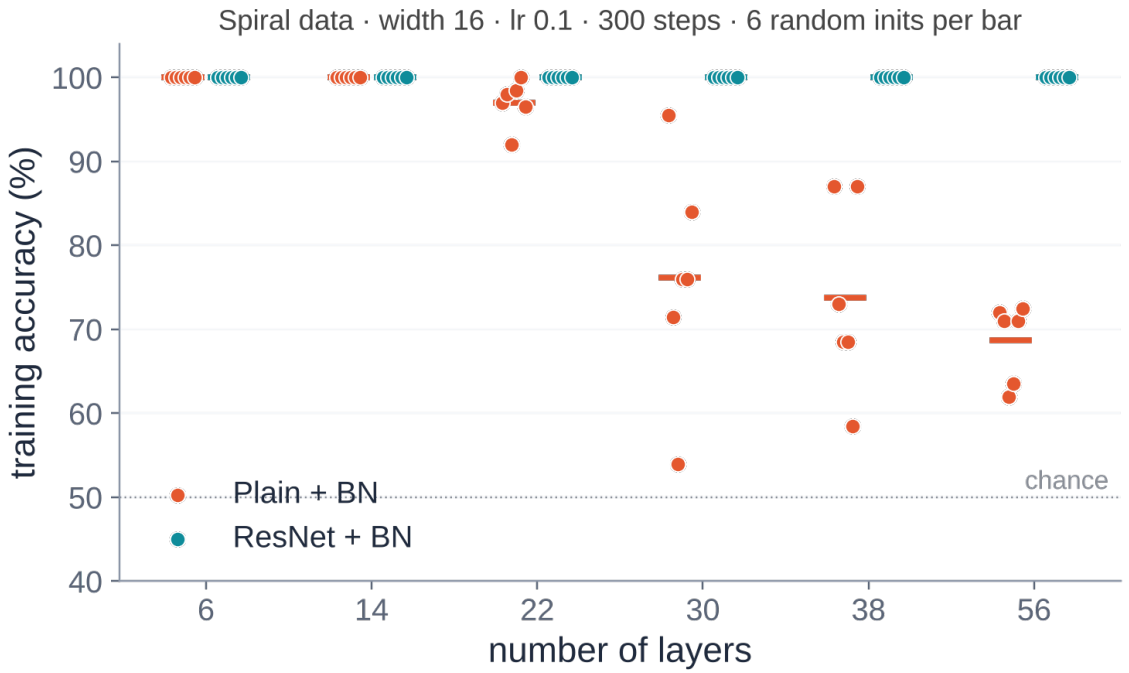
# Not Only the Last 36: Every Layer

Solution 1 gave the identity only to the 36 **new** layers. Why only those?




- Proposal of modification: Add identity mapping to each of the 56 layers

# Compare ResNet and Plain Net for Different Layers




**OUR EXPERIMENT** Lab 1 MLPs · spiral data · lr 0.1 · 300 steps

- Deeper plain + BN:

  accuracy **drops** and depends on the seed (30 layers: 54–96%)

- ResNet + BN:

  100% at every depth, every seed

Same pattern as He et al. on CIFAR-10: the skip connection improves training.

// PART 02

# Initialization & Normalization of ResNet

Why standard initialization fails for ResNet,
and how normalization layers rescue it.

02



# Roadmap

**2.1 Signal Propagation in ResNet**

Standard initialization explodes; how to scale the branches

**2.2 ResNet + Normalization**

Where to put the normalization layer, and why it works

PREDICT FIRST



# Signal Propagation in ResNet

Consider an $L$-layer linear ResNet with width $d$:

$$y = (I + W^L) \cdots (I + W^1)\, x, \qquad W^\ell \in R^{d \times d}$$

OUR EXPERIMENT $L = 100$, $d = 100$, standard init $W_{ij}^\ell \sim N(0, 1/d)$, random $x$:

**$\|y\| / \|x\| \approx 10^{15}$**

Surprising?

Shouldn't ResNet be much better than a plain net?

What special trick makes He et al.'s deep ResNets work?

COMPUTE

# Exercise: One Number, Two Minutes

$$y=(1+w_L)\cdots(1+w_1)\,x, \qquad w_\ell \sim N(0,1)$$

How big is $E[y^2] / x^2$ after 40 blocks? Write one number.

THINK



# 1D Case: Signal Explosion

Take $d = 1$:

$$y = (1 + w_L) \cdots (1 + w_1)\, x, \qquad w_\ell \sim N(0,1)$$

Answer:

$$E[y^2] = (1+1)^L x^2 = 2^L x^2$$

At L = 40: $2^{40} \approx 1.1 \times 10^{12}$. The signal explodes.

How to fix the 1D case?

# 1D Case: The Fix

**Change the variance:** pick $w_\ell \sim N(0, c)$.

$$E[y^2] = (1 + c)^L x^2$$

**Baseline:** $c = 0$

Then $E[y^2] = x^2$

**Any c ≤ 1/L works**

$(1 + \frac{1}{L})^L \leq e = O(1)$



# Signal Propagation: Result

**Claim 1.** Let $W \in R^{d \times d}$ have independent entries with $E[W_{ij}] = 0$, $Var[W_{ij}] = \gamma/d$. Then

- (i) for any fixed $x$: $E \parallel (I + W)x \parallel^2 = (1 + \gamma) \parallel x \parallel^2$
- (ii) for random $x$ independent of $W$: $E \parallel (I + W)x \parallel^2 = (1 + \gamma)\, E \parallel x \parallel^2$

**Corollary 1.** If all $W_{ij}^{\ell}$ are independent, then $E \parallel (I + W^L) \cdots (I + W^1)x \parallel^2 = (1 + \gamma)^L \parallel x \parallel^2$.

**Corollary 2.** If $\gamma \leq 1/L$, then $E \parallel (I + W^L) \cdots (I + W^1)x \parallel^2 \leq e \parallel x \parallel^2$.



# Multi-Layer Branch

What if each residual block has $m$ layers? (e.g., $m = 3$ in ResNet-50)

$$y = (1 + u_m^L \cdots u_1^L) \cdots (1 + u_m^1 \cdots u_1^1)\, x$$

We want $Var(u_m^i \cdots u_1^i) = 1/\mathrm{L}$, so choose

$$Var(u_j^i) = L^{-1/m}$$

→ Equivalently: multiply a standard initialization by $L^{-1/(2m)}$.



# Lesson: Know Why You Succeed

Branch scale controls signal propagation in ResNet.

Historically, ResNet works without special initialization to contronl signal, why?

Because BatchNorm exists already, which helps control signal in ResNet!

→ **Lesson**: whenever you design new architecture, **check signal propagation**.

Otherwise you may “succeed without knowing why”.

Such analyses now exist for many architectures (transformers, RNNs, graph nets, …).



# Roadmap

## 2.1 Signal Propagation in ResNet

Standard initialization explodes; how to scale the branches

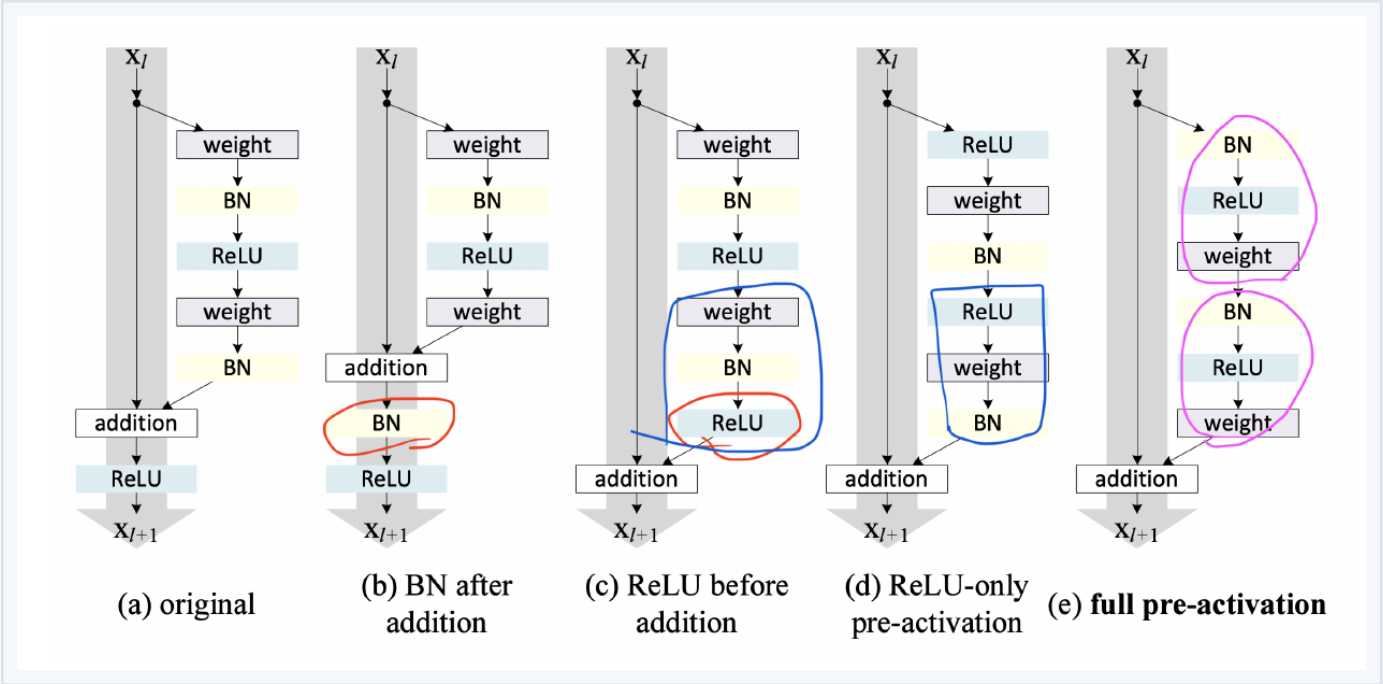
## 2.2 ResNet + Normalization

Where to put the normalization layer, and why it works

VOTE

# Various BN / ReLU Placement

FROM PAPER He et al. 2016b · ResNet-110 on CIFAR-10 · five block designs, one paper




CIFAR-10 test error, ResNet-110: (a) 6.61% (b) **8.17%** (c) 7.84% (d) 6.71% (e) 6.37%

**Five different combinations of ReLU, BN and weight**

**What do you observe about the five architectures?**

# What is similar for the five architectures

## BN always before ReLU

ReLU → W → BN.

Each of the five architectrues uses this order (or similar order, e.g., W → BN → ReLU )

Normalize the pre-activation

## Two cycles for one skip connection

Each block has rougly two (BN, ReLU, W) cycles, for one skip connection

## Differences

1) Among the three componentions BN, ReLU and weight, which starts in each branch;

2) Which of the two cycles of BN-ReLU-W are outside the branch.

One order is never used.  Which one?

REVEAL



# Kaiming He Did Not Try This

Kaiming's paper did not mention the combination:

$W \rightarrow ReLU \rightarrow BN$:

normalize the post-activation

This is still used in other places… Later lectures

// PART 02

# Loss Landscape: Visualization

03

Is training a neural net hard or easy?
Visualize the landscape, then study the theory.

# So Far: How to Train a Neural Net

## Representation

## Optimization

- **Basic**: **Gradients and backpropagation**
- **Signal propagation**: Initialization · normalization · skip connections
- **Next: loss landscape** ← today

## Generalization



# Historical Background

Because of non-convexity, convergence to bad local minima was considered a major challenge.

**Minsky & Papert**, Perceptrons (expanded edition, 1980's):

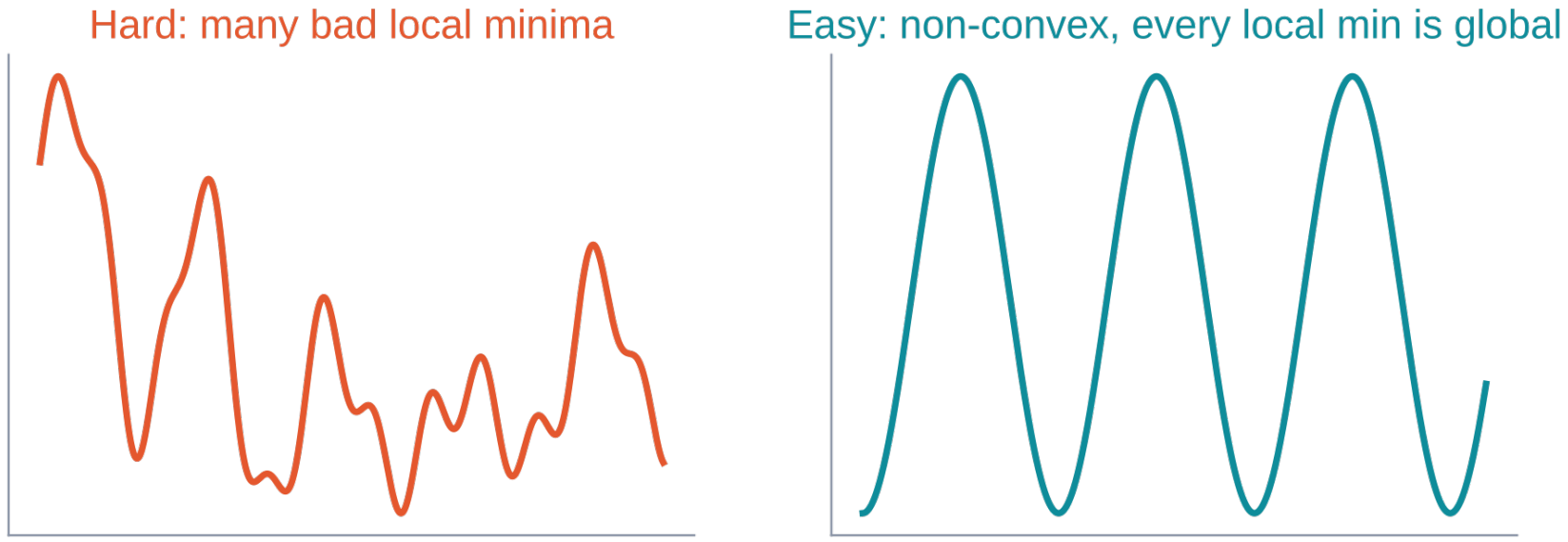
> "many people … have assumed instead that Backpropagation is a new learning scheme that somehow gets around the basic limitation of hill-climbing"

> "they speak as though becoming trapped on local maxima were rarely a serious problem"

**Bianchini & Gori '96 survey:** to address Minsky and Papert's comment, it is "very interesting to investigate the presence of local minima".

# Hard or Easy?

**VOTE:** Both are non-convex. Which one has sub-optimal local minima?




Main question: where do neural nets lie, on the easy side or the hard side?
And why?

# Hard vs Easy: Three Views

## Optimization

- Best-known boundary: convex vs non-convex (Boyd's book)
- But it is vague: some non-convex problems can be rewritten as convex ones

**Guess: NN problems are "tractable"**

## Theoretical CS

- Best-known boundary: P vs NP
- Also fine-grained complexity: $O(n^3)$ vs $O(n^2)$

**Guess: NN training can be solved efficiently (polynomial time)**

## Learning theory

- Time and sample complexity to **learn** a function
- Mixes optimization and generalization

**Guess: NN can learn many pracitcal function classes in polynomial time and space, e.g., image → labels**

**Remark**: All these "guesses" are very difficult theoretical questions, if we want rigorous proofs.
Many top researchers work on it, but still far from understanding it



# Why Is Learning Not Too Hard?

Why do connectionists (Hinton, LeCun, Bengio, …) believe learning is not too hard, despite non-convexity, NP-hardness proofs, and the challenge of generalization?

## Bottom line: humans can do it!

### Size

Human brain: ~86 billion neurons, ~$10^{14}$ synapses

Largest models: ~$10^{12}$ parameters (WuDao 2.0: 1.75T)

### Training

Millions of years of evolution

Billions of humans interacting with the environment

→ Guess: efficient algorithms for intelligence exist (no bad local min, no overfitting, …).



# How to Visualize?

Landscape is a geometric subject, so visualization helps.

Two common ways:

### Graph

$(w, f(w))$ for $w$ in an interval (1D) or

a box (2D)

### Contour

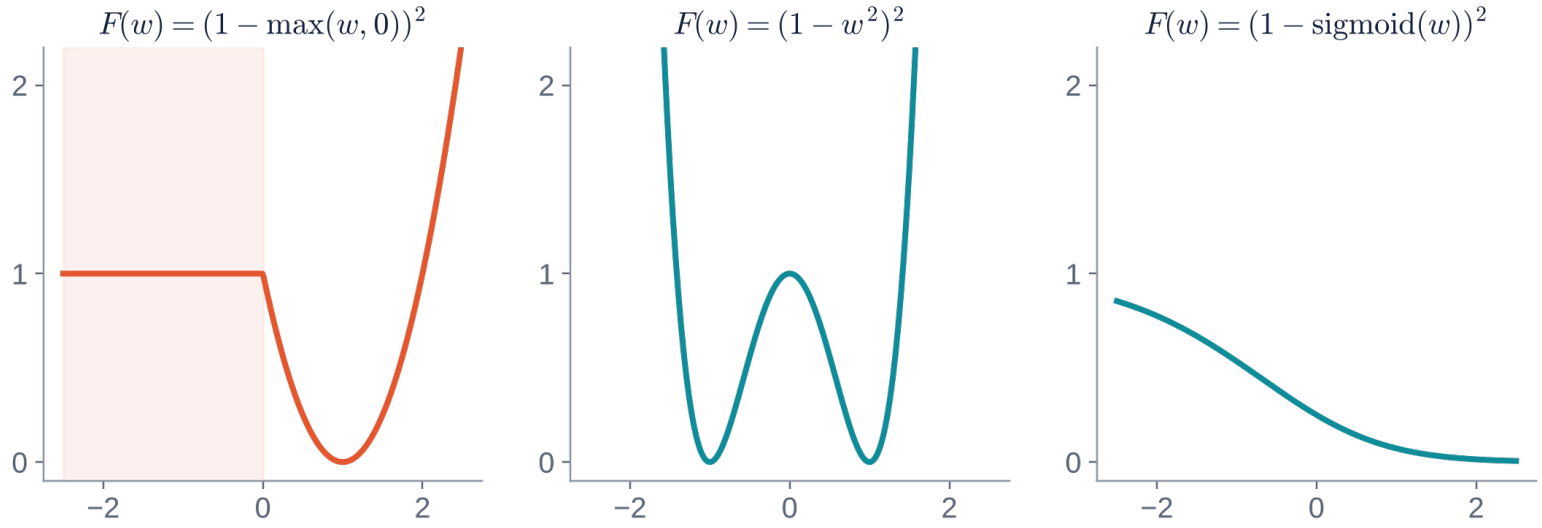
$w: f(w) = c$ for various values of $c$

Simplest case: one neuron, one sample.

$f(x; w) = \phi(wx)$, $F(w) = (y - \phi(wx))^2$; with $x = y = 1$:

$$F(w) = (1 - \phi(w))^2$$

# One Neuron: Landscape Depends on the Activation




ReLU: flat region of sub-optimal local minima

quadratic: no sub-optimal local min

strictly monotone: no sub-optimal local min

- ReLU: infinitely many sub-optimal local minima, but no sub-optimal **strict** local min
- Strictly monotone or quadratic: no sub-optimal local min



# Lesson on ReLU

→ ReLU is not always the best: its flat region can cause **bad local minima**.

## Training a GAN?

Try leaky ReLU ($max(0.1x, x)$) or tanh: no flat region

## Training a transformer?

Common choice: GELU, a smooth version of ReLU



# High Dimensions?

Visualizing a landscape in **high dimension** seems difficult.

Any way to visualize it?

# Dimension reduction!

# 2D Visualization

**Given:** a center point $\theta_0$ and two directions $u$, $v$. **Plot:**

$$F(\theta_0 + \alpha u + \beta v), \qquad \alpha, \beta \in [-1,1]$$

**VOTE:** Which surface can appear in each case—and what can we conclude?

### Strict local min

Small $\| u \|, \| v \|$

### Convex

### Non-convex

Large $\| u \|, \| v \|$

# 2D Visualization

**Given:** a center point $\theta_0$ and two directions $u$, $v$. **Plot:**

$$F(\theta_0 + \alpha u + \beta v), \qquad \alpha, \beta \in [-1,1]$$

**VOTE:** Which surface can appear in each case—and what can we conclude?

### Strict local min

Small $\| u \|, \| v \|$: a single basin

### Convex

Only a single basin

### Non-convex

Large $\| u \|, \| v \|$: perhaps multiple basins

Warning: NOT vice versa.

One basin on this slice does not imply a high-dimensional local min or convexity.

# 2D View of a Classical Non-convex Function

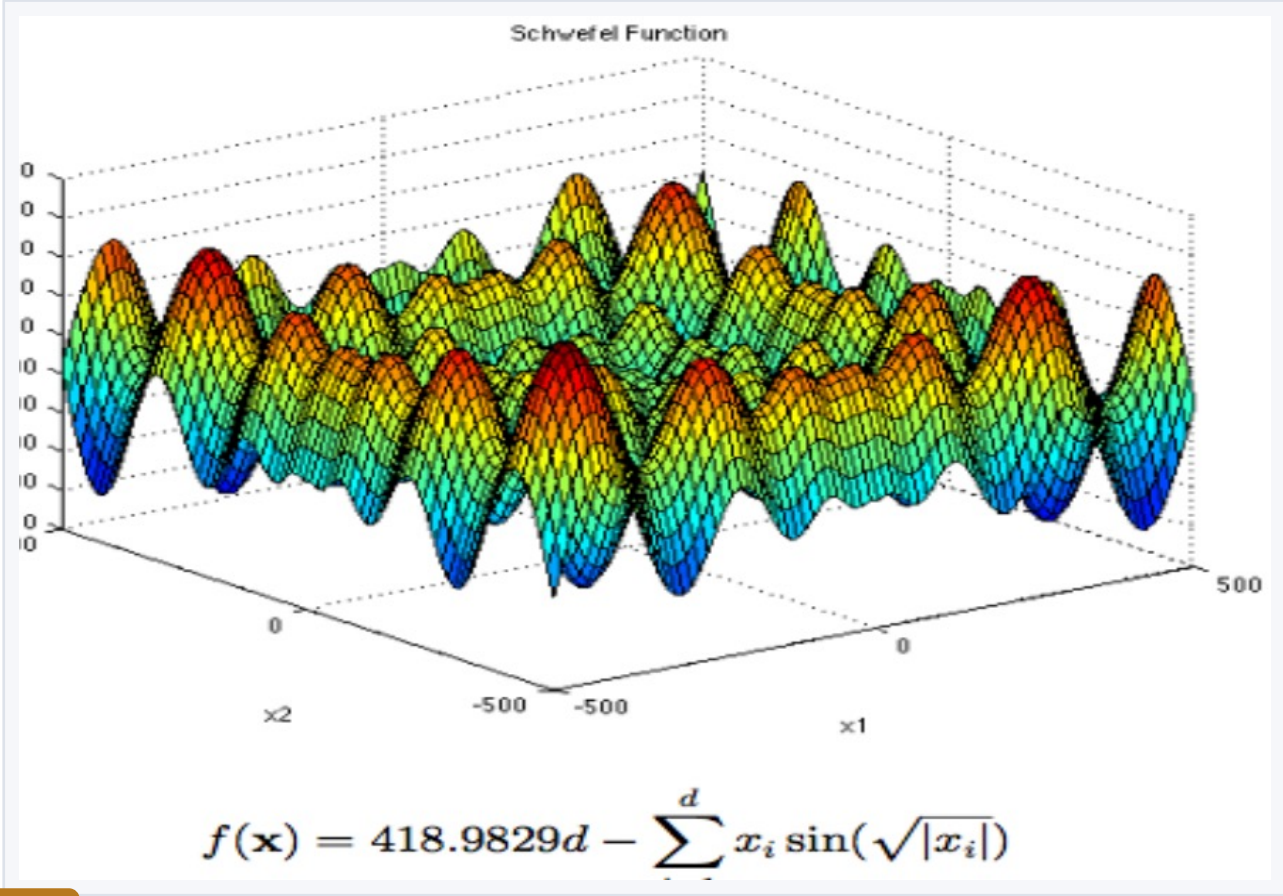



ILLUSTRATION Schwefel function of 2 variables: a standard test function with many local minima



# 2D Visualization of Neural Nets

**Center: converged solution θ***

**Directions: random, but scaled, how?**

Challenge: scale invariance.

- With BN statistics fixed, positive rescaling leaves the function unchanged
- With ReLU, scale one layer up and the next down

**Filter normalization:** draw random $u$ with the same shape as $\theta^*$; for every filter (neuron) $i$ of layer $l$:

$$u_{l,i} \leftarrow \frac{u_{l,i}}{\|u_{l,i}\|} \| \theta^*_{l,i} \|$$

Same for $v$.

TERMS **filter**: in a CNN, one output channel of a conv; in an MLP, one neuron, i.e. one row of $W_l$

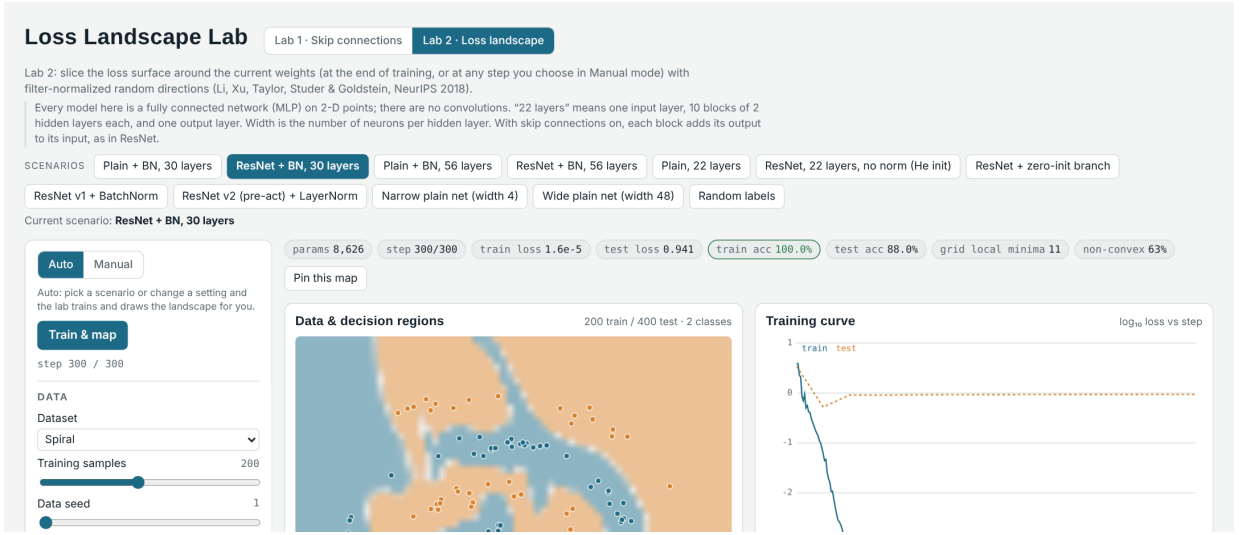
Li, Xu, Taylor, Studer, Goldstein, “Visualizing the Loss Landscape of Neural Nets”, NeurIPS 2018.

HANDS-ON

# Lab: Loss Landscape Lab

- Each run trains an MLP, then draws $L(\theta^* + \alpha\delta + \beta\eta)$ with filter-normalized directions
- **Auto**: pick a scenario; it trains and maps.
- **Manual**: draw the map at any training step
- **Pin this map** to compare maps side by side

“Lab” tab **https://ruoyus.github.io/me2605/demos/loss-landscape-lab/**



HANDS-ON

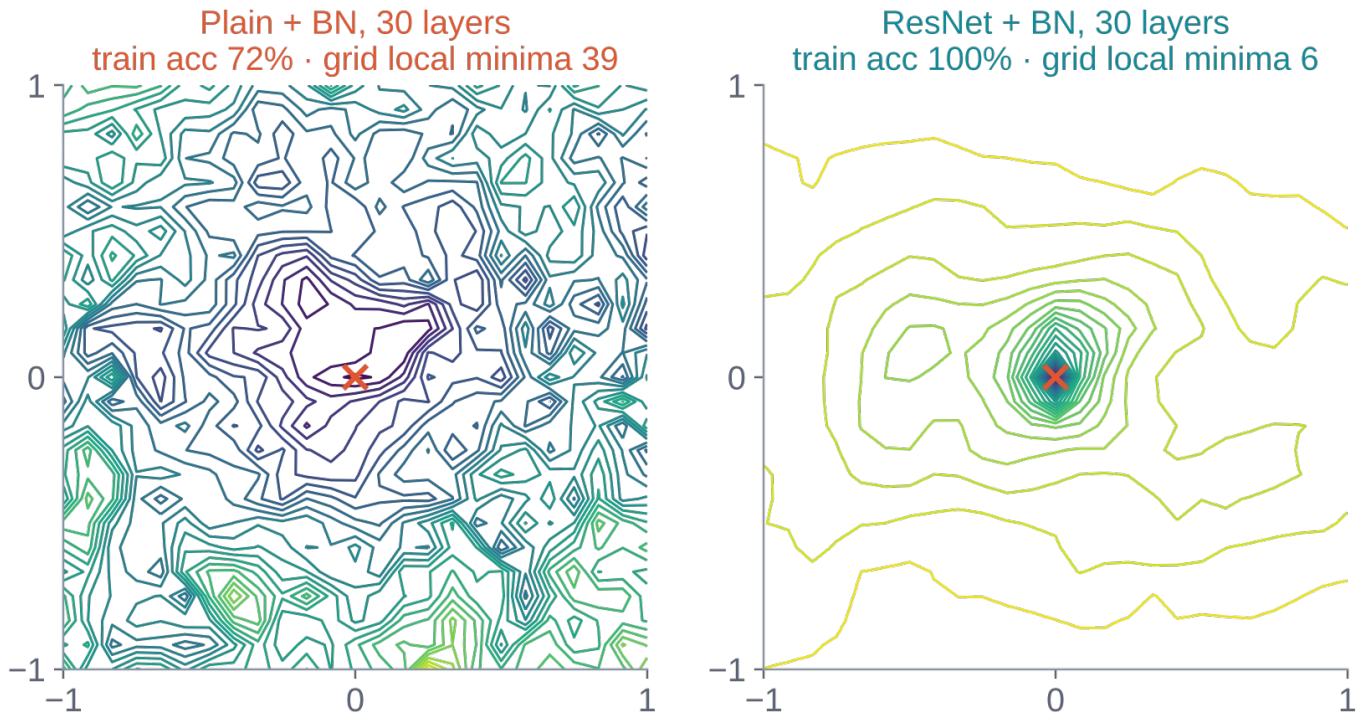


# Task 1: Skip Connection

1. Click “Plain + BN, 30 layers”, set **Init seed = 2**
2. Click “ResNet + BN, 30 layers”, set **Init seed = 2**
3. Record for each: train accuracy and grid-local dips on the plotted slice

**Q: Which map is smoother? Which net is trained better?**

# Task 1: What You Should See




OUR EXPERIMENT Lab 2 · 30-layer MLPs · Init seed 2

- Plain + BN: rugged map, many small dips; stuck at 72%
- ResNet + BN: one clean basin; 100%
- The observations are consistent

VOTE



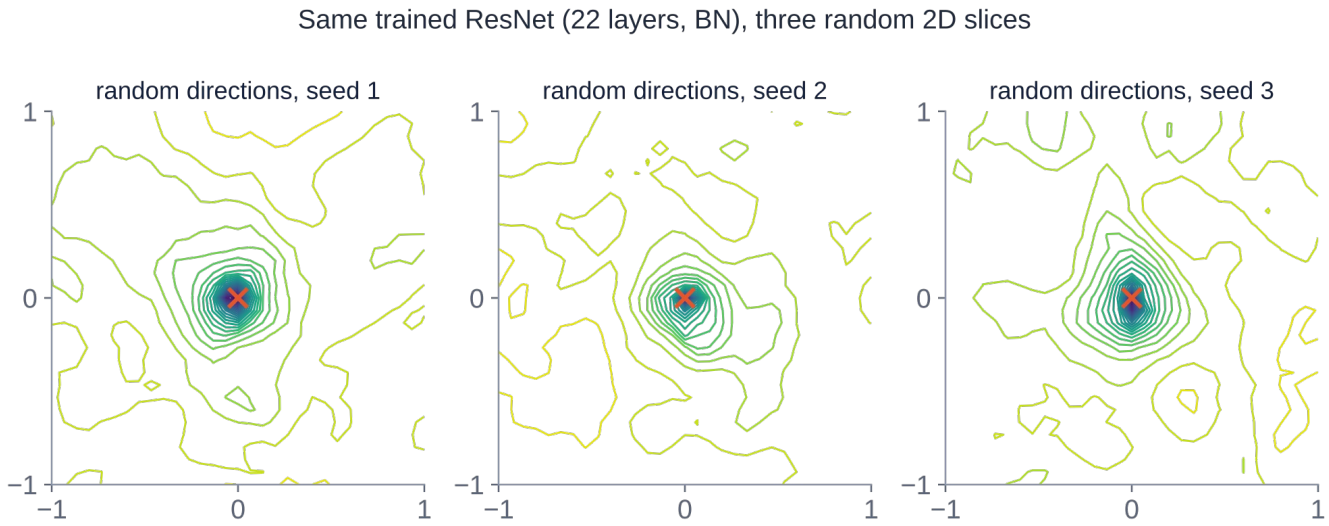
# Question 1 (answer after exploring)

> **In Task 1, the ResNet map shows one smooth basin.**
>
> **Does this show that the ResNet has no bad local minima?**
>
> **Give at least one reason.**

# Answer to Question 1

**Not a rigourous.** A 2D slice is one random 2-dim subspace of a ~5,000-dim parameter

OUR EXPERIMENT




- New random directions → different slices of one network

→ One basin on a plotted slice does NOT rigorously prove it is a local min of the original high-dimensional function;

but it is an empirical evidence that the wide-net landscape is “better”

HANDS-ON



# Task 2: Width and Multiple Initializations

1. Click “Narrow plain net (width 4)”; set **Init seed** to 1, 2, 3, 4
2. Do the same for “Wide plain net (width 48)”
3. Compare the final train accuracy of every run

**Q: Does the narrow net reach the same result every time?**
**The wide net?**

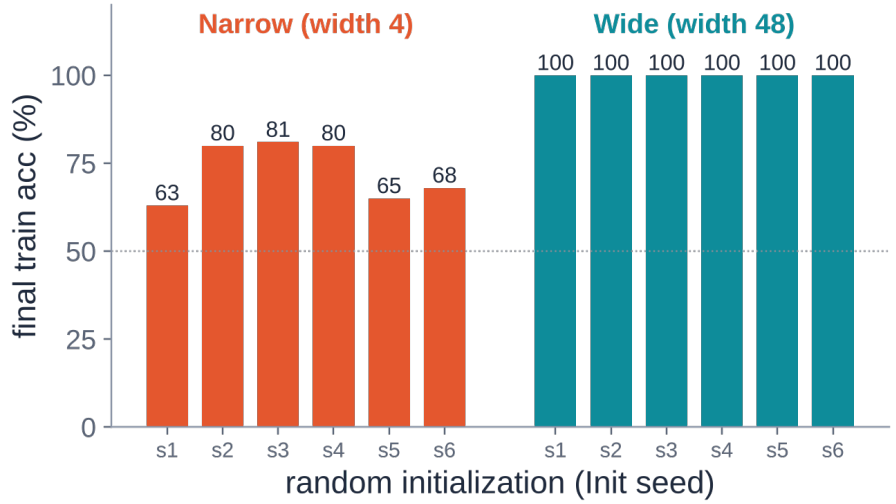
VOTE



# Question 2 (answer after exploring)

> **In Task 2, the narrow net ended between 63% and 81%, depending on the seed.**
>
> **Is that a representation error or an optimization error?**
>
> **What is your evidence?**

# Answer to Question 2




**OUR EXPERIMENT** Lab · plain MLPs, 3 blocks

- Same narrow architecture:

  81% with some init,

  63% with another init; etc.

- Wide net: 100% from every init

- So it **can** lead to at least 81% accuracy:

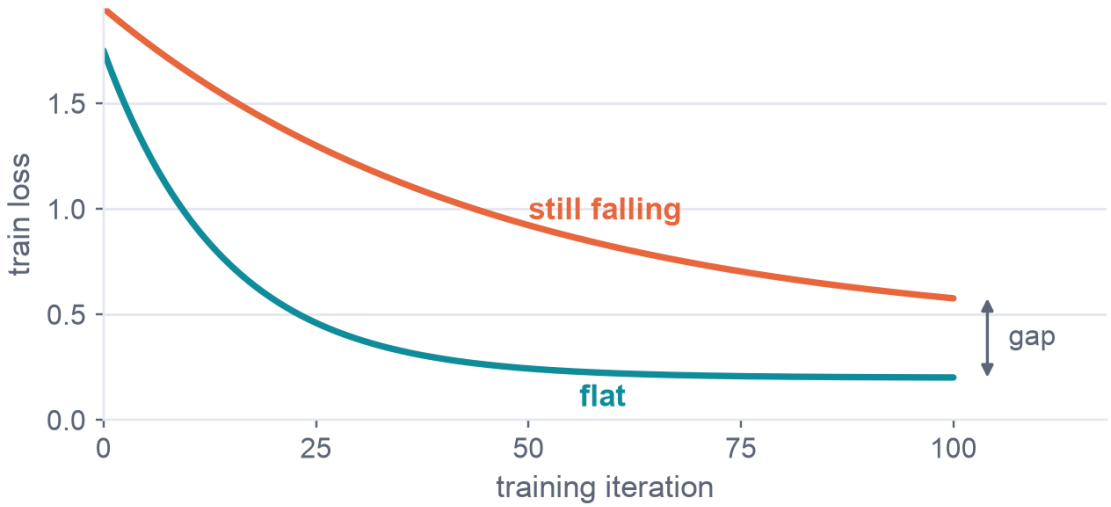
63% accuracy is due to optimization issue, not representation issue

# Different Starts, Different Results: Two Cases

Two runs, same net, same data, different initialization. If their final losses differ, y one of these holds:

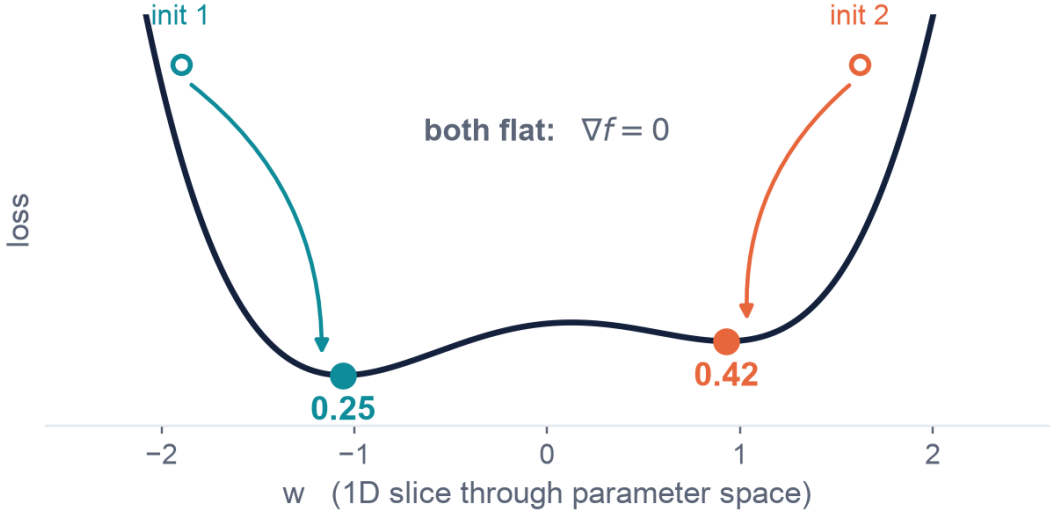
**Case 1 not every run converged**

the loss of the second run is still falling when training stops




**Case 2 all converged, to different stationary points**

two different starting points end at two different points with zero gradient




Measure $\|\nabla L\|$ at the end of each run;
if they are nearly 0, then it is Case 2:
different runs lead to different stationary points, so the global landscape is not good.

HANDS-ON



# Task 3: Why Filter Normalization?

1. Click “ResNet v1 + BatchNorm”
2. Switch **Direction scaling**:

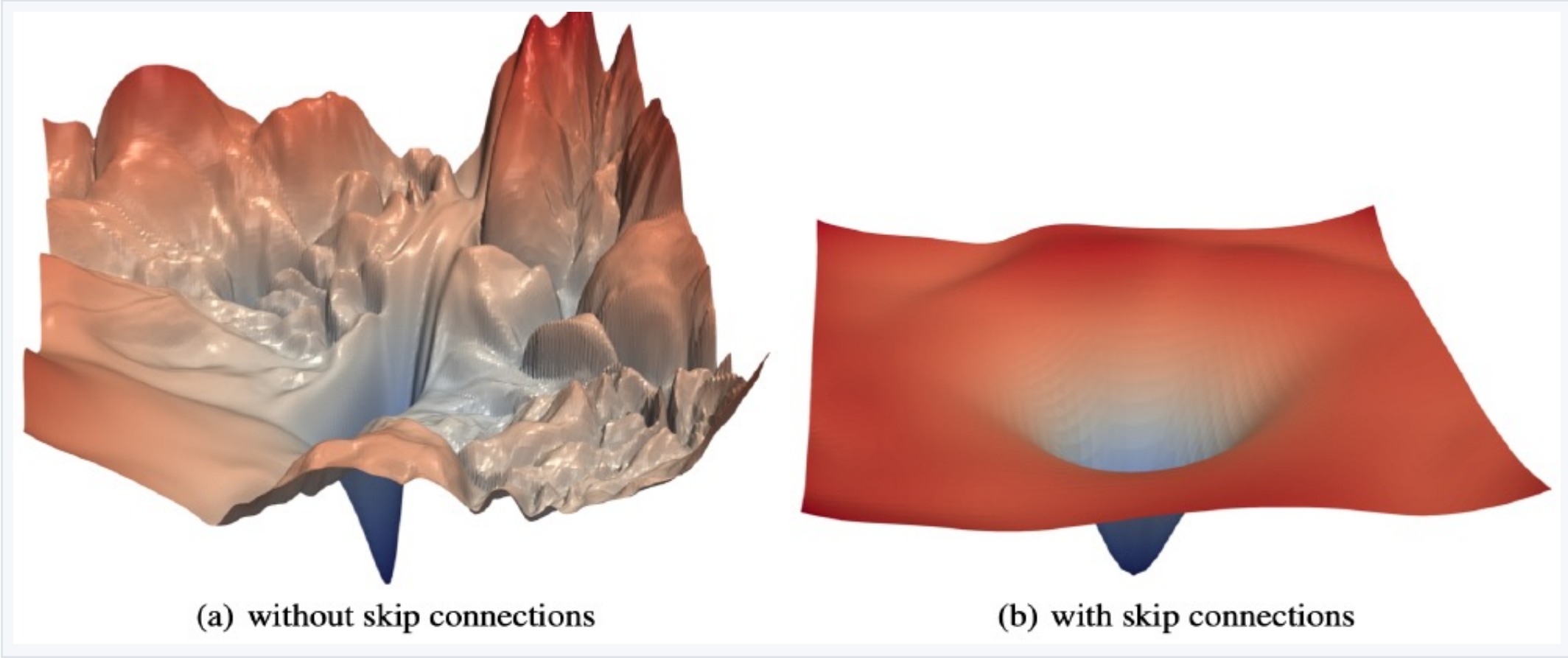
   Filter-wise → None → Layer-wise (only the map is redrawn)

1. Try another **Direction seed**

**Q: How does the map change?**

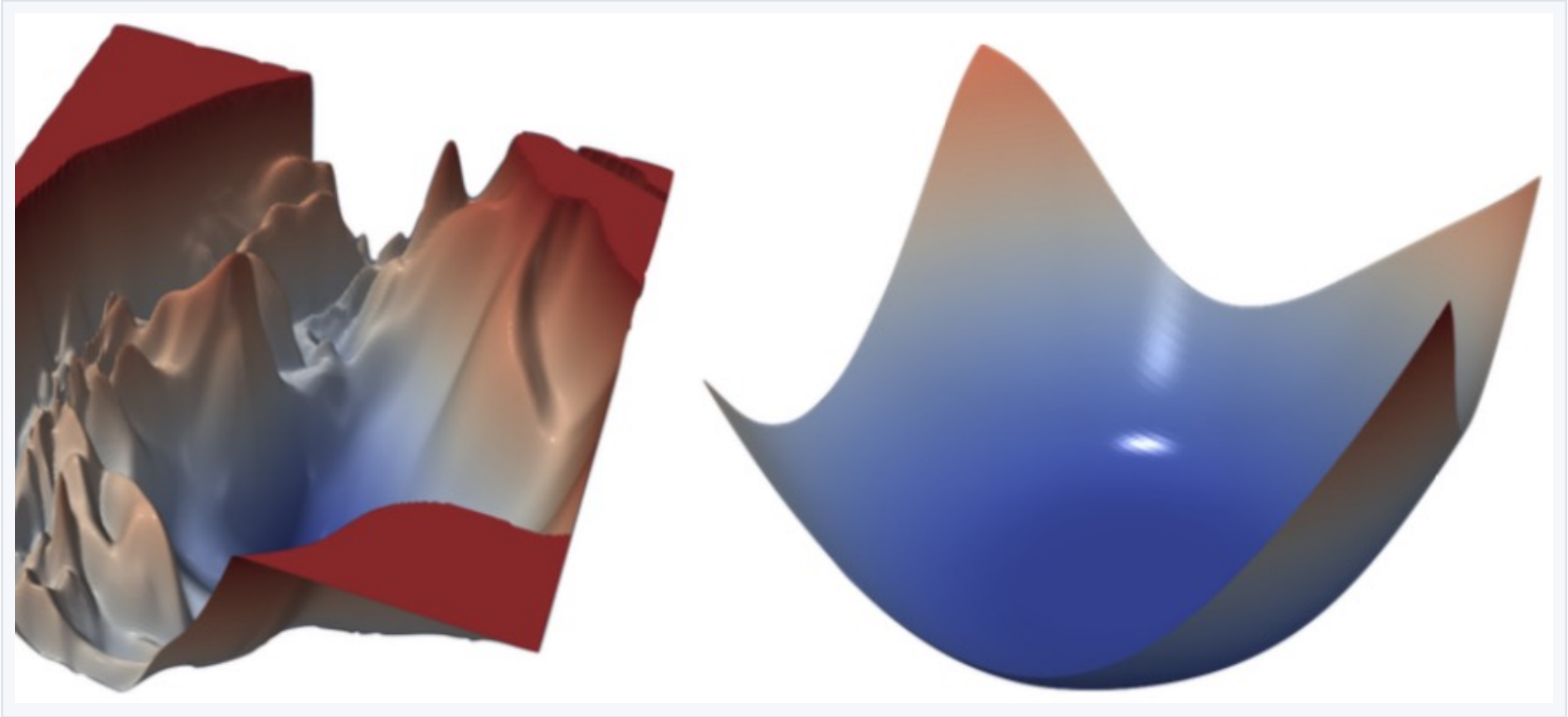
**Why are “raw” random directions (without filter normalization) possibly misleading?**

# Non-ResNet vs ResNet




FROM PAPER Li et al. 2018 · ResNet-56 (CNN) on CIFAR-10, without (left) and with (right) skip connections

# Non-ResNet vs DenseNet (an extension of ResNet)

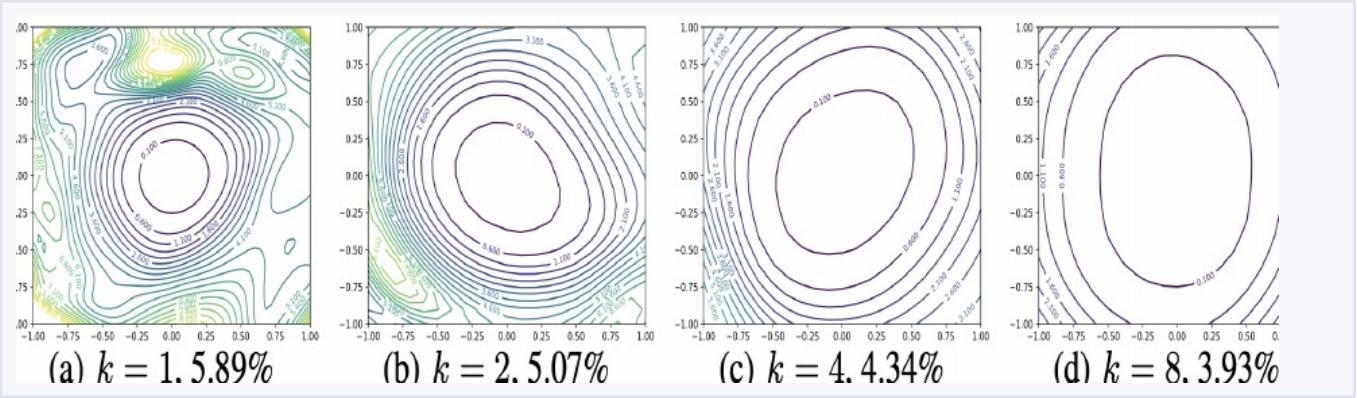


FROM PAPER Li et al. 2018 · CIFAR-10 · left: ResNet-110 without skip connections, right: DenseNet-121

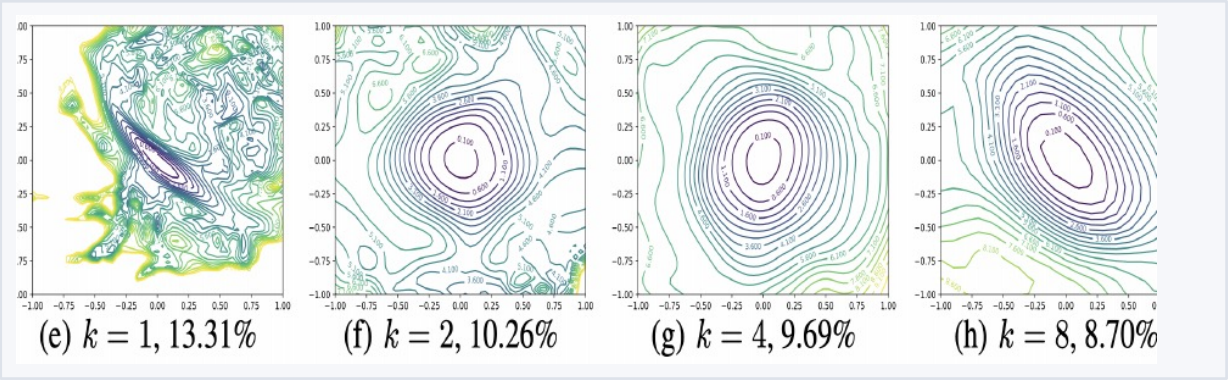
TERMS **DenseNet**: a CNN where every layer receives the outputs of all earlier layers: many skip connections

# Effect of Width

**ResNet**: width grows (k = 1 → 8), the landscape becomes rounder, the error decreases




**Non-ResNet**: same trend




FROM PAPER Li et al. 2018 · CNNs on CIFAR-10 · width multiplier k, test error under each map



# Findings of Visualization

## Finding 1: skip connections

keep the landscape smooth as depth grows

**Intuition:** an extra path removes barriers

## Finding 2: width

leads to a much smoother landscape

**Intuition:** extra paths remove barriers

// SUMMARY

# What to Take Away

// SUMMARY

# What to Take Away: Parts 1–2

**PART 1**
**Skip Connections and ResNet**

- Deep plain nets: higher **training** error, an optimization issue
- Make identity representable and add a direct path ⇒ skip connection $y = F(x) + x$

**PART 2**
**Initialization & Normalization of ResNet**

- In the 1D model: $E \| y \|^2 = (1+\gamma)^L \| x \|^2$; need $\gamma \leq \frac{1}{L}$
- Different ways to place BN, weight and ReLU

**PART 3**
**Loss Landscape: visualization**

- 2D slice: $F(\theta^* + \alpha u + \beta v)$ with **filter-normalized** directions
- Skip connections and width both make the landscape smoother
- Note: Visulization does not give a rigorous proof: a single basin ⇏ local min or convexity
- A test of landscape: whether different initialization lead to different final training error

# Thank you!